MINOR ASSIGNMENT 1- CS771

Abhishek L 230049

Soubhagya Raha 231025

Subhayan Dan 230000

Mainak Sarkar 230619

Ayush Raj 230267

Bharat Kumar 230289

Abstract

This report outlines the errors made by Melbo in implementing the C-SVM algorithm for the arbiter PUF problem. It will examine how varying the penalty parameter C affects both accuracy and convergence. Additionally, the report will analyze the impact of hyperparameters on test performance and the behavior of primal-dual convergence.

1 Task 1- Correction of Melbo's Code

On analysing the code we found errors in two places-

1.1 Wrong projection to [0, C] in doCoordOptCSVMDual function

In the function doCoordOptCSVMDual, the code to make sure that constraints are satisfied is:

```
if newAlpha < C:
    newAlpha = C
if newAlpha < 0:
    newAlpha = 0
```

But this is wrong. The constraints required to follow are 0<=newAlpha<=C, so the code should keep the values of newAlpha in the interval $[0, C]$ as it is and project the values outside this interval back into it. But this code forces every `newAlpha` that is less than C up to C, rather than projecting values outside the interval $[0, C]$ back into it.
The correct code would be:

```
if newAlpha > C:
    newAlpha = C
if newAlpha < 0:
    newAlpha = 0
```

So the logic here is that any point outside the interval $[0, C]$ should be projected to the closest point of it in the interval, which is C for newAlpha > C and 0 for newAlpha < 0.

1.2 Wrong Updatation of weights in Bookkeeping

In the function doCoordOptCSVMDual, the code written to update the primal model vector and bias values to ensure bookkeeping is proper is:

```
    w_SDCM = w_SDCM - (newAlphai + alpha[i]) * y[i] * x
    b_SDCM = b_SDCM - (newAlphai + alpha[i]) * y[i]
```

This is wrong. The correct code is:

```
    w_SDCM = w_SDCM + (newAlphai - alpha[i]) * y[i] * x
    b_SDCM = b_SDCM + (newAlphai - alpha[i]) * y[i]
```


Preprint. Under review.

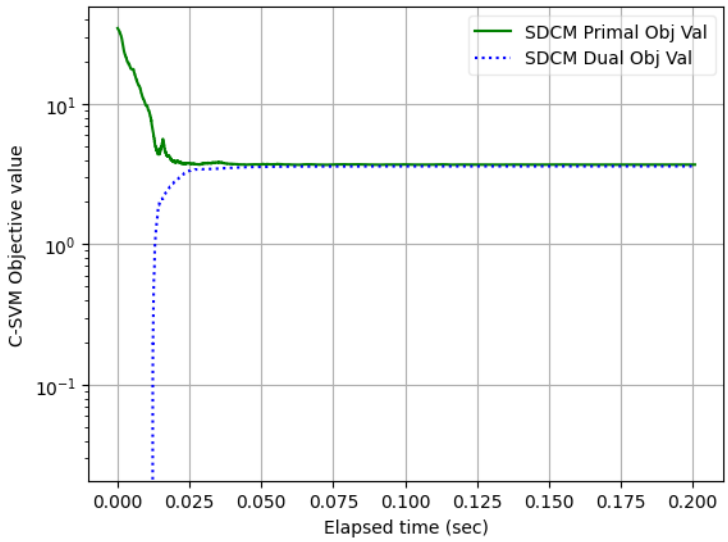


Figure 1: primal and dual objective curve

Melbo is using newAlpha+alpha instead of newAlpha[i] - alpha[i] which double-counts and moves in the wrong direction. Also, while updating weights, we are subtracting instead of adding, which moves away from the optimization.

1.3 Error in getRandpermCoord function

The current function code does not consider the 0 index at the start and immediately shifts to 1. I modified it by moving Curr=state[idx] earlier, which accounts for the 0 index. Corrected code:

```
def getRandpermCoord(state):
    idx, perm = state
    d = len(perm)
    if idx<0:
        idx=0
    curr = perm[idx]
    idx += 1
    if idx >= d:
        idx = 0
    perm = np.random.permutation(d)
    state=(idx,perm)
    return (curr, state)
```

2 Task 2-Test Accuracy and primal and dual objective curve

After correcting Melbo's error, the test accuracy I got is 0.961939393939394 or 96.1939393939394%. The primal and dual objective curves obtained are given in Figure 1.

3 Task- 3 Effect of C on test accuracy and primal and dual objective function

Refer to Table 1 and figures 2,3,4,5,6,7,8 and 9
For smaller values of C, the accuracy is low, and as I increase the value of C, as shown in Table 1, the accuracy increases. However, this trend is only observed up to 0.01. When I increase C from 0.01 to 0.1, the test accuracy starts decreasing, and as I continue increasing the value of C, the test accuracy continues to decrease. The accuracy is low at smaller C because there is more scope for misclassifications, but at larger values of C, we suffer from the problem of overfitting.
When it comes to the graph of the primal and dual objective functions, at lower C values I observe that the primal and dual objective graphs meet in less elapsed time, i.e., they converge more quickly. However, as I increase C, the time at which they meet increases. At $C = 1$, the graphs stop meeting, and at $C = 10000$, the primal and dual graphs are far from each other.

Table 1: C v/s Test Accuracy

C	Test Accuracy
0.00001	0.932070707070707
0.0001	0.932070707070707
0.001	0.9412828282828283
0.01	0.961939393939394
0.1	0.9596565656565657
1	0.9473333333333334
10	0.9371515151515152
100	0.9159191919191919
1000	0.8099292929292929
10000	0.7520808080808081

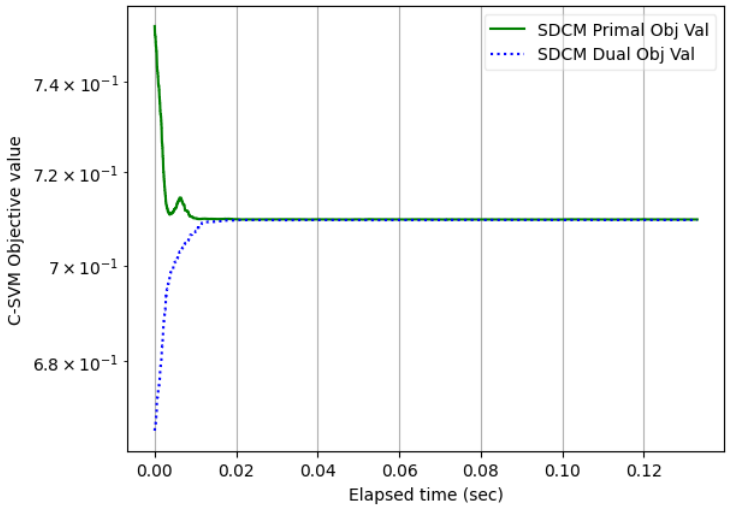


Figure 2: primal and dual objective curve for C=0.001

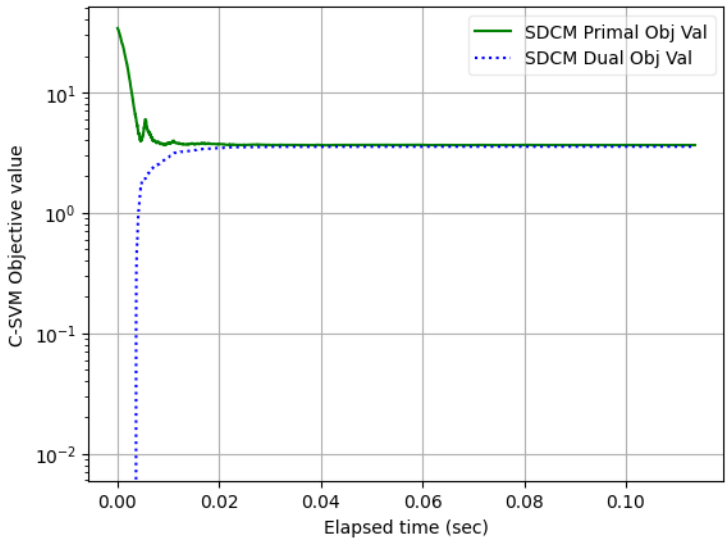


Figure 3: primal and dual objective curve for C=0.01

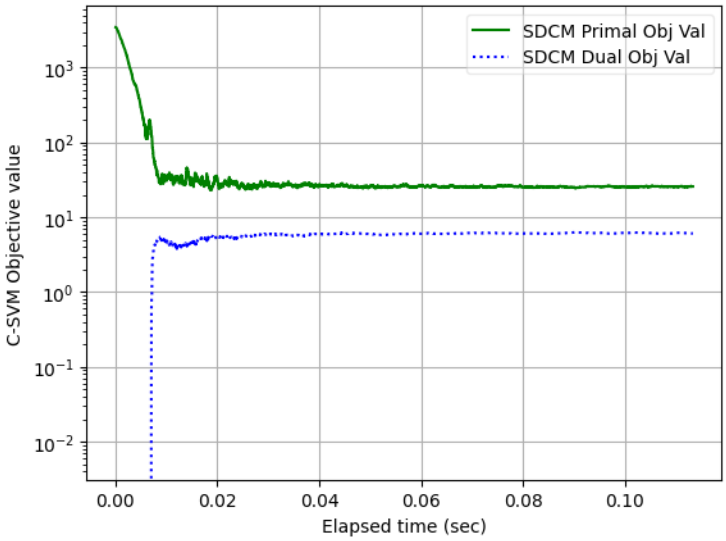


Figure 4: primal and dual objective curve for C=0.1

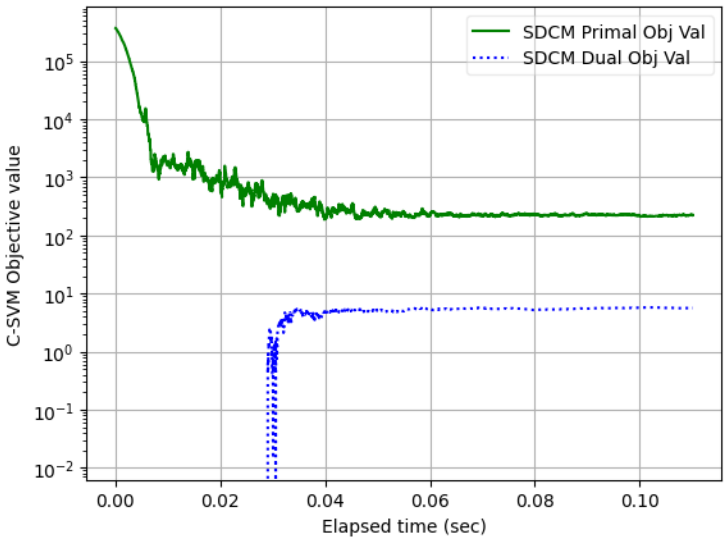


Figure 5: primal and dual objective curve for C=1

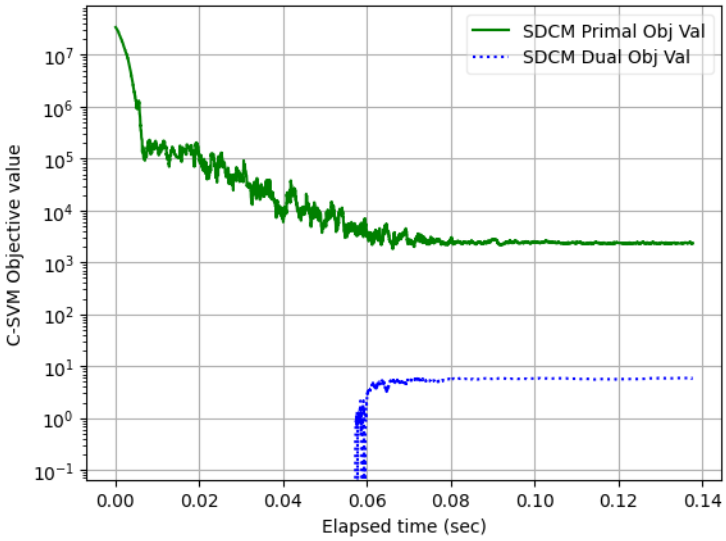


Figure 6: primal and dual objective curve for C=10

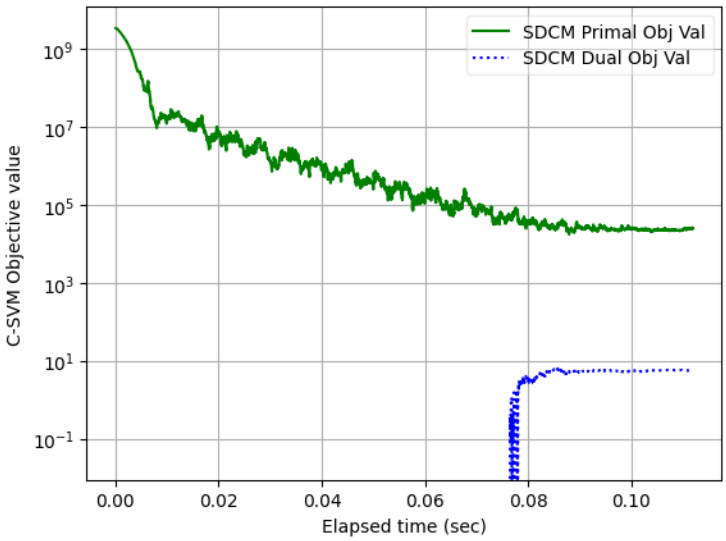


Figure 7: primal and dual objective curve for C=100

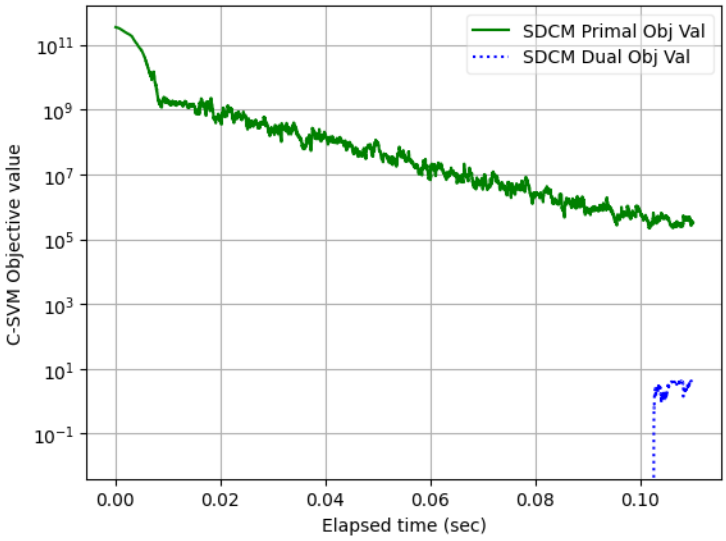


Figure 8: primal and dual objective curve for C=1000

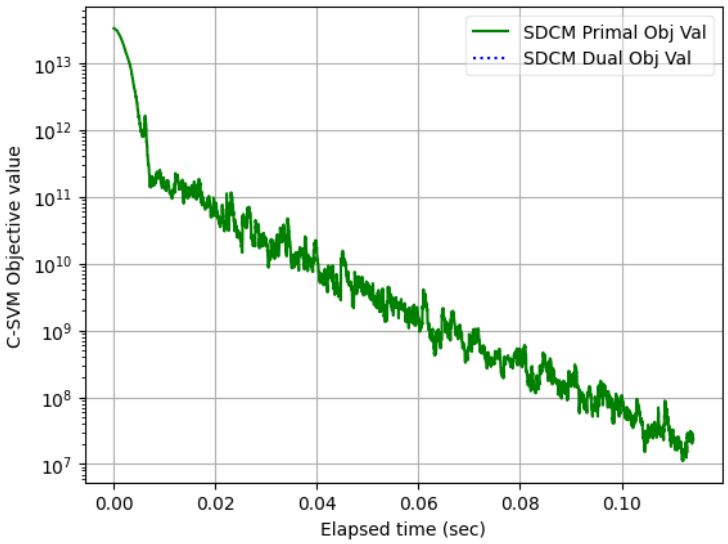


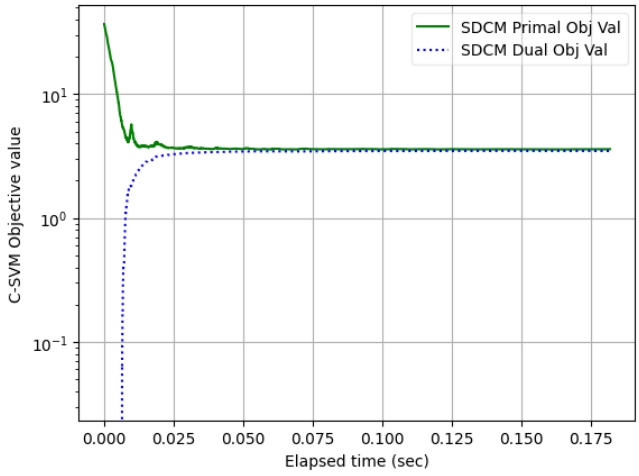
Figure 9: primal and dual objective curve for C=10000

4 Task 4-Effect of changing hyperparameters

4.1 a. Effect of changing the coordinate generator.

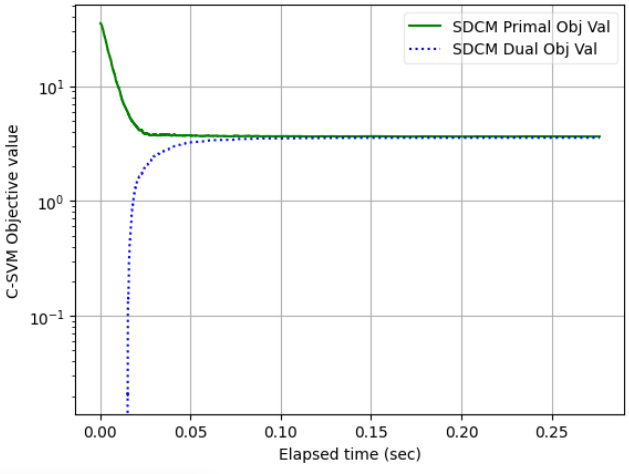
After changing the coordinate generator in different things like cyclic,random and randperm we see change in both convergence behaviour and final test accuracy. Below we report the observed effects of using *cyclic*, *random*, and *randperm* coordinate selection.

- **Cyclic:** Coordinates are selected in a fixed order. This gives stable convergence.As the required function was not present in the original code, we wrote it ourselves.



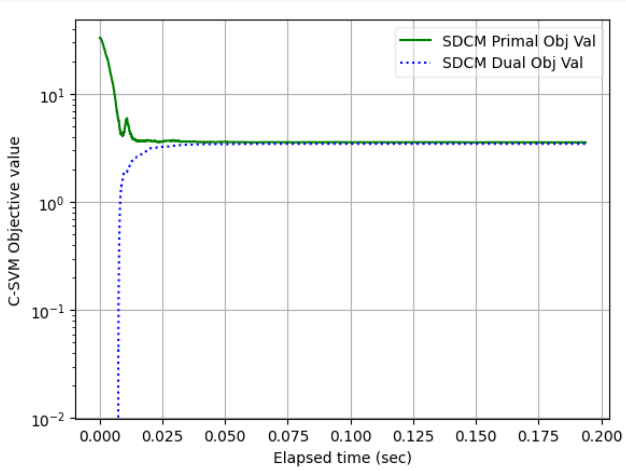
Test accuracy: 0.9476060606060606

- **Random:** At each iteration, a random coordinate is chosen. This helps us sometimes to avoid poor update patterns.As the required function was not present in the original code, we wrote it ourselves.



Test accuracy: 0.9507171717171717

- **Randperm:** A random permutation is generated and all coordinates are updated once per cycle. This balances both fairness and randomness.



Test accuracy: 0.9673939393939394

In summary, *cyclic* ensures predictability but may converge slower, *random* provides variability but unstable accuracy trends, while *randperm* often achieves a balance between fairness and randomness.

4.2 e.Random Chance

As I run the entire Jupyter notebook from start to finish multiple times, I notice variations in the test accuracy scores, as shown below. This fluctuation occurs because the train-test split randomly selects data each time, resulting in different training and testing datasets for each run, making the model different each time.

Run number	Test Accuracy
1	0.9523030303030303
2	0.960050505050505
3	0.9619393939393943
4	0.9441111111111111
5	0.9483333333333334
6	0.9593838383838383

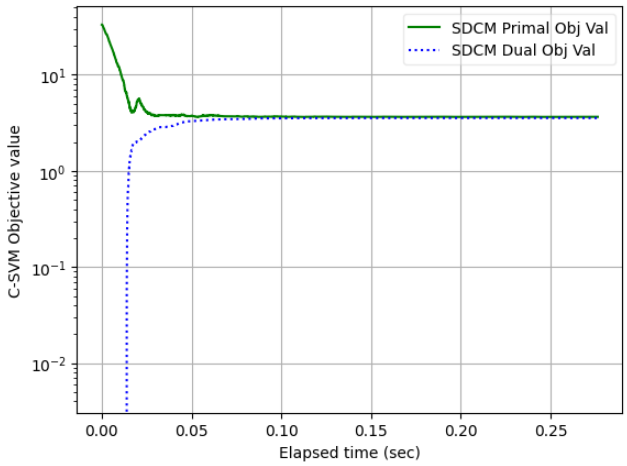
4.3 b. Effect of changing the initialization initDual.

All the above convergence curves and test accuracies were obtained with the initialization

$$\texttt{initDual} = C \cdot \mathbf{1},$$

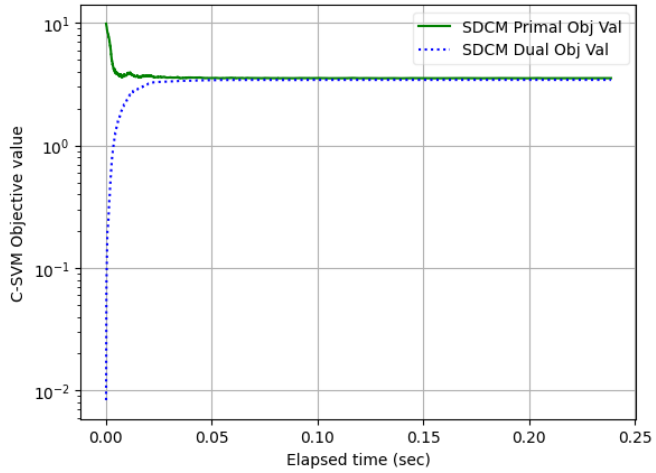
where all elements are equal to $C > 0$. Now we examine the effect of using different initialization schemes: *all-zero*, *random*, and *all-C*.I will check using the randperm coordinate generator function for all of them.

- **All-C (baseline)(C=0.01):** Every coordinate starts with the same value C. This provides a balanced initialization and often serves as the default.



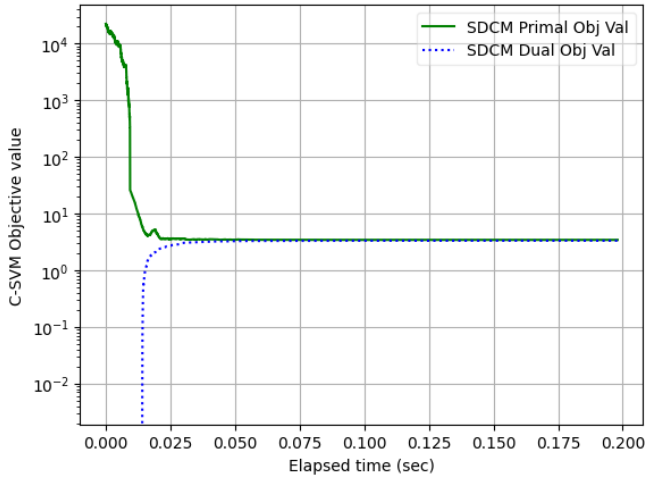
Test accuracy:0.9408181818181818

- **All-zero:** Every coordinate starts at zero. This can slow down convergence since the algorithm must build up weights from scratch.



Test accuracy: 0.9533434343434344

- **Random:** Coordinates are initialized using `numpy.random`, e.g., uniform or normal distribution. This may accelerate convergence in some cases, but stability depends on the randomness.

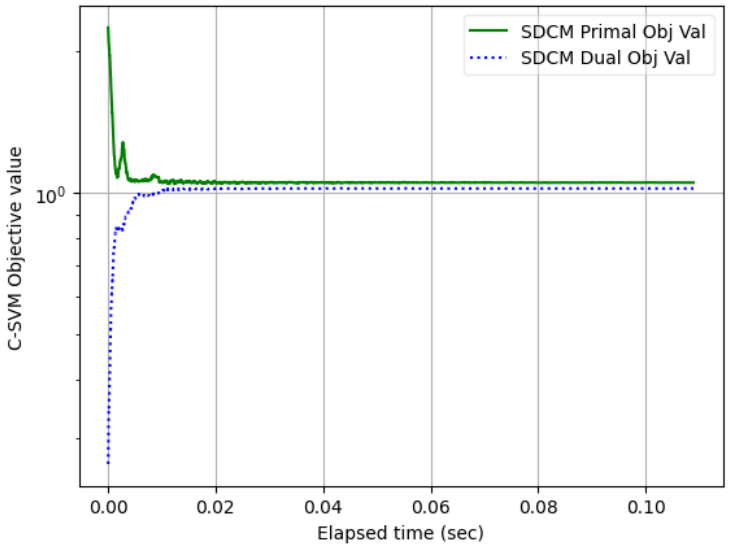


Test accuracy: 0.9478383838383838

In summary, the initialization choice affects both convergence speed and final performance. The *all-C* initialization generally provides a strong baseline, while *all-zero* may lead to slower convergence, and *random* initialization introduces variability in both convergence and accuracy.

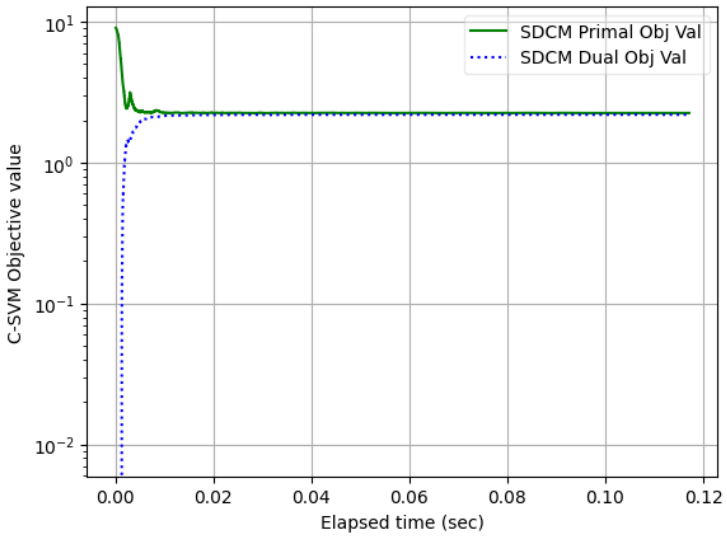
4.4 Effect of number of training points

4.4.1 Number of training points=200



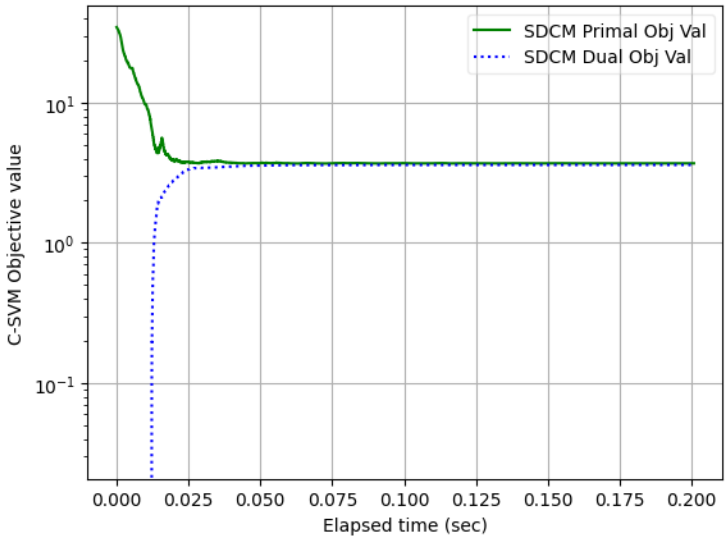
Test accuracy: 0.865561122244489

4.4.2 Number of training points=500



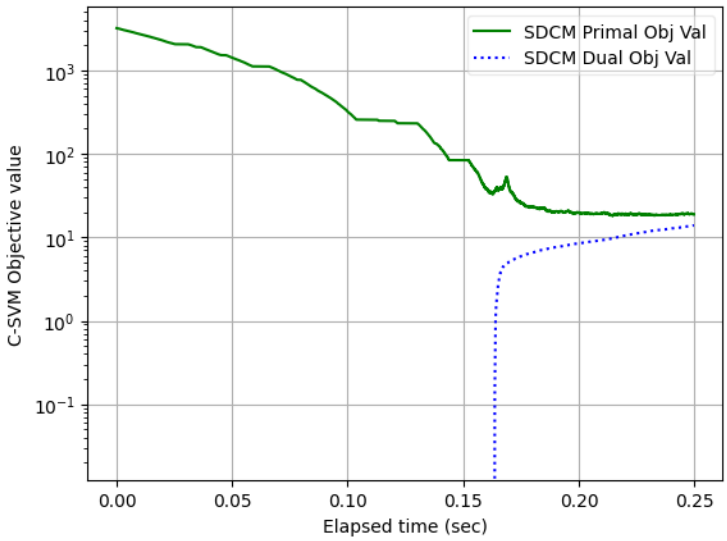
Test accuracy: 0.9248844221105528

4.4.3 Number of training points=1000



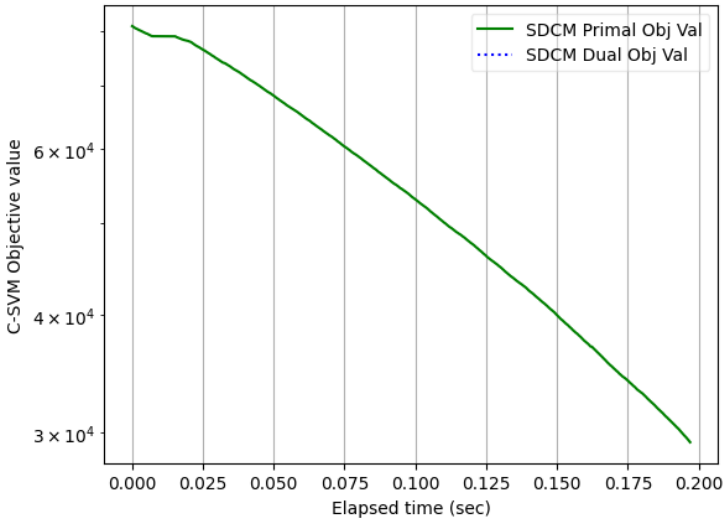
Test accuracy: 0.961939393939394

4.4.4 Number of training points=10000



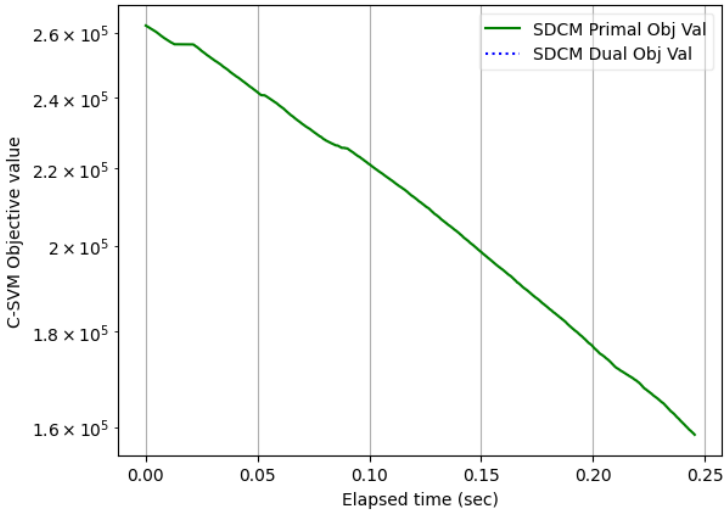
Test accuracy: 0.9791888888888889

4.4.5 Number of training points=50000



Test accuracy: 0.98958

4.4.6 Number of training points=90000



Test accuracy: 0.9904

4.4.7 Observation and reasoning

As the number of training points increases, we observe that accuracy improves; however, the time required for the primal and dual curves to converge also lengthens. This is because a larger dataset allows the model to fit the data better, resulting in increased accuracy. However, the training time for each epoch also rises, which delays the convergence of the primal and dual curves.